Sensitivity-Allocated Block Quantization for Matched-Filter Readout: A CPU Reference Study

Lee Si Yuan
Independent Researcher
siyuanlee.cs@gmail.com

Abstract

Muir introduces a matched-filter readout gain that scales the query before reading its recurrent state. We study a deployment extension that quantizes this gain in fixed-size channel tiles under a fixed nominal bit budget. Our greedy allocator gives the next bit to the tile with the largest reduction in a conservative output-error bound. For a CPU reference recurrence with 128 state rows, 32 value channels, 8-row tiles, and 64 controlled synthetic seeds, sensitivity allocation reduced mean output ℓ_2 error by 77.1% relative to uniform four-bit tile quantization at the same gain-code bit count. An analytic per-tile bound held in every trial and a Lean 4 proof checks its weighted-sum form. This is a synthetic feasibility result: scale metadata is excluded from the bit budget, and we measure neither trained-model quality nor GPU latency.

1 Introduction

Muir uses a channel-wise matched-filter gain on the query to calibrate how recurrent state rows contribute to output (Gangavarapu et al., 2026). We ask whether a quantized readout gain should use the same precision in every channel tile. We propose sensitivity-allocated block quantization: spend a fixed nominal gain-code budget where quantization has the largest effect on the readout. This is an implementation study and does not alter the learned gain or recurrent update.

No earlier Muir research artifact or extension appears in the papers collection or paper-specific Git history. The automated daily index had already captured the paper's title and abstract; we read the full arXiv v1 text for this extension. The source is Gangavarapu et al. (2026).

2 Method

For query coordinates q_j, state rows $S_j \in \mathbb{R}^{d_v}$, and learned gains γ_j, the readout is

$$y = \sum_j \gamma_j q_j S_j. \tag{1}$$

Partition channels into tiles G_t. In tile t with $b_t \geq 2$ signed quantization bits, set $L_t = 2^{b_t-1} - 1$ and $a_t = \max_{j \in G_t} |\gamma_j| / L_t$. Symmetric round-to-nearest quantization has maximum absolute error at most $a_t/2$ in exact arithmetic. For a conservative implementation bound, we use the looser radius $\max_{j \in G_t} |\gamma_j| / L_t$, which also covers ties and floating arithmetic.

By the triangle inequality and Cauchy–Schwarz,

$$\|\hat{y} - y\|_2 \leq \sum_t \max_{j \in G_t} |\hat{\gamma}_j - \gamma_j| \, \|q_{G_t}\|_2 \, \|S_{G_t}\|_F \tag{2}$$

$$\leq \sum_t \frac{\max_{j \in G_t} |\gamma_j|}{2^{b_t-1} - 1} \, \|q_{G_t}\|_2 \, \|S_{G_t}\|_F. \tag{3}$$

The CPU allocator starts each tile at two bits and spends the remaining nominal gain-code budget one bit at a time. At each step it increments the tile with the largest reduction in Eq. 3. This greedy rule is not claimed globally optimal. Scale metadata and allocation-computation cost are excluded from the nominal bit budget.

3 CPU Reference Study

We generate controlled synthetic cases with independent fixed seeds. Each case has 128 rows, value width 32, and 16 tiles of 8 rows. Tile gain ranges and state-row energies are drawn over log-uniform scales to create heterogeneous sensitivity. We compare uniform four-bit quantization with adaptive tile widths constrained to an average

Method	Mean error	Mean bound	Violations
Uniform, 4b	2.3682	50.5491	0/64
Allocated, 2–8b	0.4362	16.4972	0/64

Table 1: Synthetic CPU reference results.

of four bits per gain (512 gain-code bits per vector), excluding scale metadata, over 64 seeds. Every output error is checked against Eq. 3. Mean measured error fell 77.1% under adaptive allocation in these synthetic cases. This does not establish model-quality improvement or speedup. The bound is conservative. The CPU script does not model scale metadata, fused-kernel overhead, quantized arithmetic, or accelerator latency.

4 Formal Check

The accompanying Lean 4 file proves that per-coordinate error radii bound any nonnegative weighted signed sum, and that the peak-over-levels bound is monotone in the level count. The proof checks algebraic inequalities used here. It does not prove the measurements, allocator optimality, quantizer execution, or hardware behavior.

5 Limitations and Next Experiment

The study uses synthetic vectors and states, not Muir checkpoints, language data, or learned readout gains. The gain-code budget omits scale values, alignment, tile descriptors, and extra read bandwidth. The available host has no NVIDIA GPU, CUDA runtime, or accelerator compiler. A next experiment should use released model activations, include scales in an actual storage format, and benchmark a fused quantize/read kernel against full-precision and uniform-quantized baselines on target hardware while measuring task quality.

Reproducibility

Run `python3 experiment.py` in this directory. The script uses only the Python standard library, fixes seeds 20261109–20261172, writes `results.json`, and asserts that all 64 errors satisfy the bound. The Lean check is in `papers/formal/MuirReadoutClaims.lean`.

References

Tushaar Gangavarapu, Gil Tabak, and Clemens JS Schaefer. 2026. Muir: Linear Attention with Sensitivity-Allocated Gating and Matched-Filter Readout. arXiv:2611.08376v1. https://arxiv.org/abs/2611.08376v1.